

Generalized Linear Model in R

Md. Mostakim

Student of Applied Statistics and Data Science
Institute of Applied Statistics and Data Science
University of Dhaka



Section 1

What this presentation covers

What this presentation covers

This presentation introduces the main ideas and practical steps involved in fitting and interpreting Generalized Linear Models.

1. Recall linear model foundations

- ▶ Review the ordinary linear model.
- ▶ Fit models using `lm()`.
- ▶ Fit an equivalent Gaussian model using `glm()`.

2. Understand the GLM framework

- ▶ Understand the random component.
- ▶ Understand the systematic component.
- ▶ Learn about link functions.
- ▶ Understand the role of the `family` argument.

3. Fit models for different response variables

- ▶ Continuous outcomes using the Gaussian family.
- ▶ Count outcomes using the Poisson family.
- ▶ Binary outcomes using the Binomial family.

Section 2

Refresher on fitting linear models

Refresher on fitting linear models

```
data("ChickWeight")
lm(formula = weight ~ Diet, data = ChickWeight)
```

```
##
## Call:
## lm(formula = weight ~ Diet, data = ChickWeight)
##
## Coefficients:
## (Intercept)      Diet2      Diet3      Diet4
##      102.65      19.97      40.30      32.62
```

```
# Fit a glm()
glm(formula = weight ~ Diet , data = ChickWeight, family = 'gaussian')
```

```
##
## Call:  glm(formula = weight ~ Diet, family = "gaussian", data =
↪ ChickWeight)
##
## Coefficients:
## (Intercept)      Diet2      Diet3      Diet4
##      102.65      19.97      40.30      32.62
##
## Degrees of Freedom: 577 Total (i.e. Null);  574 Residual
## Null Deviance:      2915000
## Residual Deviance: 2759000  AIC: 6546
```

Section 3

Poisson Regression

In this exercise, you will fit a Poisson regression using `glm()`. GLMs in R use the same formula notation as linear models: `response ~ predictor`. However, the model also requires the error or distribution family to be specified with the `family = ...` argument. To fit a Poisson regression, set family equal to `"poisson"` (make sure to include the quotes " around poisson).

You will use a simulated dataset with a predictor variable `time` and response `count`.

Fitting a Poisson regression in R (cont.)

```
time <- c(
  1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
  11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
  21, 22, 23, 24, 25, 26, 27, 28, 29, 30
)

count <- c(
  0, 0, 0, 0, 1, 0, 0, 1, 0, 0,
  2, 0, 1, 0, 0, 1, 0, 0, 0, 2,
  2, 1, 1, 4, 1, 1, 1, 1, 0, 0
)

dat <- data.frame(time, count)
```

```
# fit y predicted by x with data.frame dat using the poisson family
poisson_out <- glm(count~time,
                   data = dat,
                   family = "poisson")

# print the output
print(poisson_out)
```

```
##
## Call:  glm(formula = count ~ time, family = "poisson", data = dat)
##
## Coefficients:
## (Intercept)          time
##   -1.43036       0.05815
##
## Degrees of Freedom: 29 Total (i.e. Null);  28 Residual
## Null Deviance:      35.63
## Residual Deviance: 30.92    AIC: 66.02
```

Comparing linear and Poisson regression

```
# Fit a glm with count predicted by time using data.frame dat and gaussian  
↪ family  
lm_out <- glm(count~time,  
               data = dat,  
               family = "gaussian")  
print(summary(lm_out))
```

```
##  
## Call:  
## glm(formula = count ~ time, family = "gaussian", data = dat)  
##  
## Coefficients:  
##              Estimate Std. Error t value Pr(>|t|)  
## (Intercept)  0.09425    0.32891   0.287   0.7766  
## time        0.03693    0.01853   1.993   0.0561 .  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for gaussian family taken to be 0.7714815)  
##  
##      Null deviance: 24.667  on 29  degrees of freedom  
## Residual deviance: 21.601  on 28  degrees of freedom
```

Comparing linear and Poisson regression (cont.)

```
## AIC: 81.283
```

```
##
```

```
## Number of Fisher Scoring iterations: 2
```

```
print(summary(poisson_out))
```

```
##
## Call:
## glm(formula = count ~ time, family = "poisson", data = dat)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.43036    0.59004  -2.424   0.0153 *
## time         0.05815    0.02779   2.093   0.0364 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 35.627  on 29  degrees of freedom
## Residual deviance: 30.918  on 28  degrees of freedom
## AIC: 66.024
##
## Number of Fisher Scoring iterations: 5
```

Question

Given the two models you fit, `lm_out` and `poisson_out`, which model had a statistically significant trend (using $p < 0.05$ as the criteria for statistically significant)?

- a) The linear model (GLM with the Gaussian family).
- b) The Poisson regression (GLM with the Poisson family).

Answer: ✓ b.

This exercise will show you how R's formulas allow two types of intercepts to be estimated. You will use data for the number of goals per game for two players, "Scoring Sam" and "Unlucky Lou". Because these are counts, use a `glm()` with a Poisson family.

First, fit a model that estimates an intercept and the difference between the two players. This is the default formula option in R and looks like `formula = response ~ intercept`.

Second, fit a model that estimates the effect (or intercept) of both players. This formula option in R requires a `-1`, for example `formula = response ~ intercept - 1`.

```
player <- c(
  "Sam", "Sam", "Sam", "Sam", "Sam",
  "Lou", "Lou", "Lou", "Lou", "Lou"
)

goal <- c(
  1, 2, 0, 4, 3,
  0, 0, 1, 0, 0
)

scores <- data.frame(player, goal)
```

Fit a glm() that estimates the difference between players

```
summary(glm(formula = goal ~ player,  
            data = scores,  
            family = "poisson"))
```

```
##  
## Call:  
## glm(formula = goal ~ player, family = "poisson", data = scores)  
##  
## Coefficients:  
##             Estimate Std. Error z value Pr(>|z|)  
## (Intercept)  -1.6094      0.9999  -1.610   0.1075  
## playerSam     2.3026      1.0487   2.196   0.0281 *  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for poisson family taken to be 1)  
##  
##      Null deviance: 18.3578  on 9  degrees of freedom  
## Residual deviance:  9.8105  on 8  degrees of freedom  
## AIC: 26.682  
##  
## Number of Fisher Scoring iterations: 5
```

```
# Fit a glm() that estimates an intercept for each player
```

```
summary((glm(goal ~ player - 1,  
             data = scores,  
             family = 'poisson')))
```

```
##  
## Call:  
## glm(formula = goal ~ player - 1, family = "poisson", data = scores)  
##  
## Coefficients:  
##             Estimate Std. Error z value Pr(>|z|)  
## playerLou  -1.6094      0.9999  -1.610   0.1075  
## playerSam   0.6931      0.3162   2.192   0.0284 *  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for poisson family taken to be 1)  
##  
##      Null deviance: 18.4546  on 10  degrees of freedom  
## Residual deviance:  9.8105  on  8  degrees of freedom  
## AIC: 26.682  
##  
## Number of Fisher Scoring iterations: 5
```

As you can see, R allows for two different types of intercepts to be estimated. The first and default setting estimated the number of goals scored by Sam and the difference in goals scored by Lou. The second setting, using -1 , estimated the number of goals scored by Sam AND the number of goals scored by Lou. More generally, the default formula in R uses the first factor level and contrasts all other levels to it. Reordering a factor allows you to change the reference level.

Section 4

More generally

Model 1: With an intercept

The model is:

$$\log(\mu_i) = \beta_0 + \beta_1 I(\text{Sam})$$

where Lou is the reference category.

From the output:

Intercept = -1.6094

playerSam = 2.3026

Interpretation

For Lou:

$$\log(\mu_{Lou}) = -1.6094$$

so

Model 1: With an intercept (cont.)

$$\mu_{Low} = e^{-1.6094} = 0.2$$

For Sam:

$$\log(\mu_{Sam}) = -1.6094 + 2.3026 = 0.6932$$

so

$$\mu_{Sam} = e^{0.6932} \approx 2$$

The coefficient playerSam is a log rate ratio:

$$\log\left(\frac{\mu_{Sam}}{\mu_{Lou}}\right) = 2.3026$$

thus

$$\frac{\mu_{Sam}}{\mu_{Lou}} = e^{2.3026} = 10$$

Sam's expected goal count is 10 times Lou's.

Model 2: No intercept

Removing the intercept gives one coefficient per level:

$$\log(\mu_i) = \beta_{Lou}I(\text{Lou}) + \beta_{Sam}I(\text{Sam})$$

Output:

$$\text{playerLou} = -1.6094$$

$$\text{playerSam} = 0.6931$$

Interpretation

Now each coefficient is directly the log mean for that player:

$$\mu_{Lou} = e^{-1.6094} = 0.2$$

$$\mu_{Sam} = e^{0.6931} = 2$$

Notice:

Model 2: No intercept (cont.)

$$0.6931 = -1.6094 + 2.3026$$

which is exactly the Sam mean from the first model.

The first model answers “**How does Sam differ from Lou?**” while the second answers “**What is the mean for each player?**”. They fit exactly the same underlying Poisson model.

Section 5

Basic `lm()` functions with `glm()`

Applying `summary()`, `print()`, and `tidy()` to glm

`print()` and `summary()` are two of the backbone R functions for examining models. These functions tell us about the basic model outputs. Using data from Louisville, KY, USA about the number of civilian fire injury victims per day, you will examine a linear regression's output and a Poisson regression's output.

In addition to the base R function, we will use the `tidy` function from the `broom` package. This creates a `tidy` output for each of the models. Note that `broom` needs to be installed and does not come with base R.

```
dat <- read.table("dat.txt", header = T)
```

```
dat$Month <- as.factor(dat$Month)
```

```
# Build your linear and Poisson regression models
```

```
lm_out <- lm(Number ~ Month, data = dat)
```

```
poisson_out <- glm(Number ~ Month, data = dat, family = "poisson")
```

```
# Examine the outputs using print()
```

```
print(lm_out)
```

```
##  
## Call:  
## lm(formula = Number ~ Month, data = dat)  
##  
## Coefficients:  
## (Intercept)      Month2      Month3      Month4      Month5  
↪ Month6  
##      0.129477    -0.038031    -0.078401    -0.057254    -0.032702  
↪ -0.043365  
##      Month7      Month8      Month9      Month10     Month11  
↪ Month12  
##     -0.005821    -0.051520    -0.023921    -0.054208    -0.023921  
↪ -0.022919
```

```
print(poisson_out)
```

```
##
## Call:  glm(formula = Number ~ Month, family = "poisson", data = dat)
##
## Coefficients:
## (Intercept)      Month2      Month3      Month4      Month5
↪ Month6
##      -2.0443      -0.3478      -0.9302      -0.5838      -0.2911
↪ -0.4079
##      Month7      Month8      Month9      Month10     Month11
↪ Month12
##      -0.0460      -0.5073      -0.2043      -0.5424      -0.2043
↪ -0.1948
##
## Degrees of Freedom: 4367 Total (i.e. Null);  4356 Residual
## Null Deviance:      2325
## Residual Deviance: 2303  AIC: 2976
```

```
# Examine the outputs using summary()
```

```
summary(lm_out)
```

```
##
## Call:
## lm(formula = Number ~ Month, data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.1295 -0.1056 -0.0914 -0.0753  8.8763
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.129477   0.022770   5.686 1.38e-08 ***
## Month2      -0.038031   0.032767  -1.161  0.2458
## Month3      -0.078401   0.032007  -2.450  0.0143 *
## Month4      -0.057254   0.032269  -1.774  0.0761 .
## Month5      -0.032702   0.032007  -1.022  0.3070
## Month6      -0.043365   0.032269  -1.344  0.1791
## Month7      -0.005821   0.032007  -0.182  0.8557
## Month8      -0.051520   0.032007  -1.610  0.1075
## Month9      -0.023921   0.032269  -0.741  0.4586
## Month10     -0.054208   0.032007  -1.694  0.0904 .
## Month11     -0.023921   0.032269  -0.741  0.4586
## Month12     -0.022919   0.032136  -0.713  0.4758
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
***
```

```
summary(poisson_out)
```

```
##
## Call:
## glm(formula = Number ~ Month, family = "poisson", data = dat)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -2.0443      0.1459 -14.015  < 2e-16 ***
## Month2        -0.3478      0.2314  -1.503  0.132839
## Month3        -0.9302      0.2719  -3.422  0.000623 ***
## Month4        -0.5837      0.2444  -2.388  0.016923 *
## Month5        -0.2911      0.2215  -1.314  0.188706
## Month6        -0.4079      0.2314  -1.763  0.077939 .
## Month7        -0.0460      0.2074  -0.222  0.824486
## Month8        -0.5073      0.2361  -2.149  0.031671 *
## Month9        -0.2043      0.2182  -0.936  0.349112
## Month10       -0.5424      0.2387  -2.272  0.023075 *
## Month11       -0.2043      0.2182  -0.936  0.349112
## Month12       -0.1948      0.2166  -0.899  0.368434
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 2325.3  on 4367  degrees of freedom
## Residual deviance: 2302.7  on 4356  degrees of freedom
## AIC: 2325.6
```

```
library(broom)
```

```
# Examine the outputs using tidy()
```

```
tidy(lm_out)
```

```
## # A tibble: 12 x 5
```

##	term	estimate	std.error	statistic	p.value
##	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
##	1 (Intercept)	0.129	0.0228	5.69	0.0000000138
##	2 Month2	-0.0380	0.0328	-1.16	0.246
##	3 Month3	-0.0784	0.0320	-2.45	0.0143
##	4 Month4	-0.0573	0.0323	-1.77	0.0761
##	5 Month5	-0.0327	0.0320	-1.02	0.307
##	6 Month6	-0.0434	0.0323	-1.34	0.179
##	7 Month7	-0.00582	0.0320	-0.182	0.856
##	8 Month8	-0.0515	0.0320	-1.61	0.108
##	9 Month9	-0.0239	0.0323	-0.741	0.459
##	10 Month10	-0.0542	0.0320	-1.69	0.0904
##	11 Month11	-0.0239	0.0323	-0.741	0.459
##	12 Month12	-0.0229	0.0321	-0.713	0.476

```
tidy(poisson_out)
```

```
## # A tibble: 12 x 5
```

##	term	estimate	std.error	statistic	p.value
##	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
##	1 (Intercept)	-2.04	0.146	-14.0	1.27e-44
##	2 Month2	-0.348	0.231	-1.50	1.33e- 1
##	3 Month3	-0.930	0.272	-3.42	6.23e- 4
##	4 Month4	-0.584	0.244	-2.39	1.69e- 2
##	5 Month5	-0.291	0.221	-1.31	1.89e- 1
##	6 Month6	-0.408	0.231	-1.76	7.79e- 2
##	7 Month7	-0.0460	0.207	-0.222	8.24e- 1
##	8 Month8	-0.507	0.236	-2.15	3.17e- 2
##	9 Month9	-0.204	0.218	-0.936	3.49e- 1
##	10 Month10	-0.542	0.239	-2.27	2.31e- 2
##	11 Month11	-0.204	0.218	-0.936	3.49e- 1
##	12 Month12	-0.195	0.217	-0.899	3.68e- 1

Extracting coefficients from glm()

Sometimes, we specifically care about model coefficients and their confidence intervals. The `coef()` function extracts coefficients and `confint()` extracts the confidence intervals. These functions work the same for both linear and generalized linear models. The GLM you fit in the previous exercise (`poisson_out`) has been loaded for you.

```
# Extract the regression coefficients  
coef(poisson_out)
```

```
## (Intercept)      Month2      Month3      Month4      Month5      Month6  
## -2.04425523 -0.34775767 -0.93019964 -0.58375226 -0.29111968 -0.40786159  
##      Month7      Month8      Month9      Month10      Month11      Month12  
## -0.04599723 -0.50734279 -0.20426264 -0.54243411 -0.20426264 -0.19481645
```

```
# Extract the confidence intervals  
confint(poisson_out)
```

```
##              2.5 %      97.5 %  
## (Intercept) -2.3444432 -1.77136313  
## Month2      -0.8103027  0.10063404  
## Month3      -1.4866061 -0.41424128  
## Month4      -1.0762364 -0.11342457  
## Month5      -0.7311289  0.14051326  
## Month6      -0.8704066  0.04053012  
## Month7      -0.4542037  0.36161360  
## Month8      -0.9807831 -0.05092540  
## Month9      -0.6367321  0.22171492  
## Month10     -1.0218277 -0.08165226  
## Month11     -0.6367321  0.22171492  
## Month12     -0.6237730  0.22851779
```

Data scientists often use models to predict future situations. GLMs are one such tool and, when used for these situations, they are sometimes called supervised learning.

For this exercise, you will predict the expected number of daily civilian fire injury victims for the North American summer months of June (6), July (7), and August (8) using the Poisson regression you previously fit and the `new_dat` dataset.

Recall that the Poisson slope and intercept estimates are on the natural log scale and can be exponentiated to be more easily understood. You can do this by specifying `type = "response"` with the `predict` function.

```
new_dat <- data.frame(Month = as.factor(c(6, 7, 8)))
```

```
# print the new input months  
print(new_dat)
```

```
##      Month  
## 1         6  
## 2         7  
## 3         8
```

```
# use the model to predict with new data  
pred_out <- predict(object = poisson_out, newdata = new_dat, type =  
  ↪ "response")
```

```
# print the predictions  
print(pred_out)
```

```
##           1           2           3  
## 0.08611111 0.12365591 0.07795699
```

Section 6

Logistic Regression

Pittsburgh is a city located in Allegheny County, PA, USA. Commuters living in this area have different options for commuting to work, including taking the bus. Using data from 2015, you will see if the number of commuting days per week increases the chance somebody will ride the bus. The choice of riding the bus is a binary outcome, hence you will use a logistic regression to model this data

```
bus <- read.csv("busData.csv")

bus$Bus <- as.factor(bus$Bus)

# Build a glm using the bus data frame that models Bus predicted by
# CommuteDays
bus_out <- glm(formula = Bus ~ CommuteDays,
               data = bus,
               family = "binomial")
```

Examining & interpreting logistic regression outputs

In the previous exercise, you fit a logistic regression, `bus_out`. During this exercise, you will examine `bus_out` and interpret the results of the regression using the tools you learned about in Chapter 1:

- ▶ `print()` includes the coefficient estimates (i.e., slopes and intercepts) for different predictor variables and information about the model fit such as deviance.
- ▶ `summary()` includes the `print()` outputs as well as standard errors, z-scores, and P-values for the coefficient estimates.
- ▶ `tidy()` includes the `summary()` coefficient table as a tidy data frame.

Recall that regression coefficients can help us understand both the direction of relationships and statistical significance of coefficients. For logistic regression, a positive coefficient indicates that the probability of an event occurring increases as a predictor increases.

Examining & interpreting logistic regression outputs (cont.)

```
print(bus_out)
```

```
##  
## Call:  glm(formula = Bus ~ CommuteDays, family = "binomial", data = bus)  
##  
## Coefficients:  
## (Intercept)  CommuteDays  
##      -1.4549      0.1299  
##  
## Degrees of Freedom: 15891 Total (i.e. Null);  15890 Residual  
## Null Deviance:      19570  
## Residual Deviance: 19540      AIC: 19540
```

Core Definitions

Null Deviance (19570): This measures how well the response variable is predicted by a model that includes only the intercept (no predictor variables). It represents the worst-case baseline error.

Residual Deviance (19540): This measures the error after adding your predictor variable (`CommuteDays`).

How to Interpret the Numbers

Model Improvement: The deviance decreased from 19570 to 19540. This reduction of 30 units ($19570 - 19540$) indicates that adding `CommuteDays` improves the model's fit over the baseline model.

Degrees of Freedom: The model used 1 degree of freedom to achieve this drop (15891 Null DF minus 15890 Residual DF), which corresponds to the single predictor variable added.

Goodness of Fit: To determine if this drop of 30 is statistically significant, you can compare it to a Chi-Square (χ^2) distribution with 1 degree of freedom. A drop of 30 on 1 degree of freedom yields an extremely small p-value ($p < 0.001$), meaning `CommuteDays` is a highly significant predictor.

```
# Look at the summary of bus_out  
summary(bus_out)
```

```
##  
## Call:  
## glm(formula = Bus ~ CommuteDays, family = "binomial", data = bus)  
##  
## Coefficients:  
##             Estimate Std. Error z value Pr(>|z|)  
## (Intercept) -1.45493    0.11471 -12.683  < 2e-16 ***  
## CommuteDays  0.12985    0.02312   5.616 1.96e-08 ***  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for binomial family taken to be 1)  
##  
##      Null deviance: 19568  on 15891  degrees of freedom  
## Residual deviance: 19536  on 15890  degrees of freedom  
## AIC: 19540  
##  
## Number of Fisher Scoring iterations: 4
```

```
# Look at the tidy() output of bus_out  
tidy(bus_out)
```

```
## # A tibble: 2 x 5  
##   term          estimate std.error statistic  p.value  
##   <chr>         <dbl>     <dbl>     <dbl>   <dbl>  
## 1 (Intercept)  -1.45      0.115     -12.7  7.32e-37  
## 2 CommuteDays    0.130     0.0231      5.62  1.96e- 8
```

Question

What is the relationship between the number of commute days and the probability of taking the bus?

- a. There is no statistically significant relationship.
- b. There is a positive significant relationship (commuting more days increases the chance someone takes the bus).
- c. There is a negative significant relationship (commuting fewer days decreases the chance someone takes the bus).

Answer: ✓ b.

Section 7

Bernoulli vs binomial

Simulating binary data

A Bernoulli distribution is a special case of a binomial. Next, you will see how to simulate both in R and then examine the outputs to see how they are similar. Both distributions can be simulated with the random binomial function: `rbinom()`. `rbinom()` requires 3 arguments:

- ▶ `n`, which is the number of draws or random numbers (i.e., an output vector of length `n`).
- ▶ `size`, which is the number of samples per draw (i.e., the maximum value for each random number).
- ▶ `prob`, which is the probability for the simulation.

To sample with a Bernoulli, you simply use `size = 1`.

If we take a single random draw (`n = 1`) from a binomial distribution with a large number of samples per draw (e.g. `size = 100`), we should get similar results as a taking a many samples (e.g. `n = 100`) with 1 sample per draw (`size = 1`).

```
# Simulate 1 draw with a sample size of 100  
binomial_sim <- rbinom(n = 1, size = 100, prob = 0.5)  
  
# Simulate 100 draw with a sample size of 1  
bernoulli_sim <- rbinom(n = 100, size = 1, prob = 0.5)  
  
# Print the results from the binomial  
print(binomial_sim)
```

```
## [1] 50
```

```
# Sum the results from the Bernoulli  
sum(bernoulli_sim)
```

```
## [1] 55
```

Long-form logistic regression input

```
# Create vector x
x <- c(rep("a", 14), rep("b", 14))

# Create vector y
y <- as.factor(c(
  "fail", "fail", "fail", "fail", "success", "fail", "fail",
  "fail", "fail", "fail", "fail", "fail", "fail", "success",
  "success", "fail", "success", "success", "success", "success", "success",
  "success", "success", "success", "success", "fail", "success", "fail"
))

data_long <- data.frame(x, y)
```

A binomial regression can take inputs in three different formats. The first requires data to be in “long” format and directly models each observation (e.g. a response of numeric 0/1 or a factor of yes/no):

x	y
a	fail
a	fail
a	fail
a	fail
a	success
a	fail

In this format, the formula response predicted by predictor (or `response ~ predictor` in R's formula syntax) is used. During this exercise, you'll fit a regression using this format.

```
# Fit a a long format logistic regression
```

```
lr_1 <- glm(y~x, data = data_long, family = "binomial")
```

```
print(lr_1)
```

```
##  
## Call:  glm(formula = y ~ x, family = "binomial", data = data_long)  
##  
## Coefficients:  
## (Intercept)          xb  
##      -1.792         3.091  
##  
## Degrees of Freedom: 27 Total (i.e. Null);  26 Residual  
## Null Deviance:      38.67  
## Residual Deviance: 26.03    AIC: 30.03
```

Wide-form input logistic regression

```
library(tidyr)

library(tidyverse)

data_wide <- data_long %>%
  count(x, y) %>%
  pivot_wider(
    names_from = y,
    values_from = n
  ) %>%
  mutate(Total = fail + success,
         successProportion = success/Total)
```

The second and third approaches for fitting logistic regressions require “wide” format data:

x	fail	success	Total	successProportion
a	12	2	14	0.1428571
b	3	11	14	0.7857143

For the second approach, model a 2 column matrix of successes and failures (e.g. number of no's and yes's per group).

In this format, use the formula `cbind(success, fail) ~ predictor`.

For the third approach, model the probability of success (e.g. group 1 had 75% yes and group 2 had 65% no) and the weight or number of observations per group (e.g. there were 40 individuals in group 1 and 100 individuals in group 2). In this example, `successProportion = success / Total`.

In this format, the formula

`proportion of successes ~ response variable` is used with `weights = number in treatment`.

```
# Fit a wide form logistic regression
lr_2 <- glm(
  formula = cbind(success, fail) ~ x,
  data = data_wide,
  family = "binomial"
)
```

```
# Print the output of lr_2
print(lr_2)
```

```
##
## Call:  glm(formula = cbind(success, fail) ~ x, family = "binomial",
##       data = data_wide)
##
## Coefficients:
## (Intercept)          xb
##      -1.792       3.091
##
## Degrees of Freedom: 1 Total (i.e. Null);  0 Residual
## Null Deviance:      12.64
## Residual Deviance: 4.441e-15      AIC: 9.215
```

```
# Using data_wide, fit a glm with successProportion  
# predicted by x and weights = Total
```

```
lr_3 <- glm(  
  formula = successProportion ~ x,  
  weights = Total,  
  data = data_wide,  
  family = "binomial"  
)
```

```
# Print the output of lr_3  
print(lr_3)
```

```
##  
## Call:  glm(formula = successProportion ~ x, family = "binomial", data =  
  ↪ data_wide,  
##      weights = Total)  
##  
## Coefficients:  
## (Intercept)          xb  
##      -1.792        3.091  
##  
## Degrees of Freedom: 1 Total (i.e. Null);  0 Residual  
## Null Deviance:      12.64  
## Residual Deviance: 4.441e-15      AIC: 9.215
```

Comparing logistic regression outputs

When building models, you want to have more observations than parameters that are estimated for the model. These extra variables are called degrees of freedom.

A model with too few observations can be overfit, or even be un-fitable (sometimes call singular). Furthermore, looking at the degrees for freedom can help you double-check your data and code. For example, a mismatch between the degrees of freedom and the number of observations you think you have can indicate that your data needs to be cleaned more, there is a bug in your code, or that there is a modeling error.

The wide versus long input formats for the `glm()` produce different degrees of freedom because the difference in number of rows of the data causes the model to think that there is a difference in number of observations.

Section 8

Probit versus logit

Probit and Logit models are both non-linear regression methods used to predict binary outcomes (0 or 1), but they use different underlying statistical distributions.

Underlying Distribution: Logit uses the logistic distribution (S-shaped curve with slightly fatter tails), while probit uses the standard normal distribution (bell curve).

Mathematical Formula: Logit has a closed-form formula ($1/(1 + e^{-z})$), making it simpler to compute. Probit relies on the cumulative distribution function (CDF) of the normal distribution, which involves an integral.

Coefficient Scaling: Logit coefficients are typically about 1.6 times larger than probit coefficients when applied to the same data, because of how their error variances are scaled.

Interpretation: Logit results convert easily into odds ratios, which are popular in medical and social sciences. Probit is often favored in economics because many latent variables are assumed to be normally distributed.

Which One to Choose?

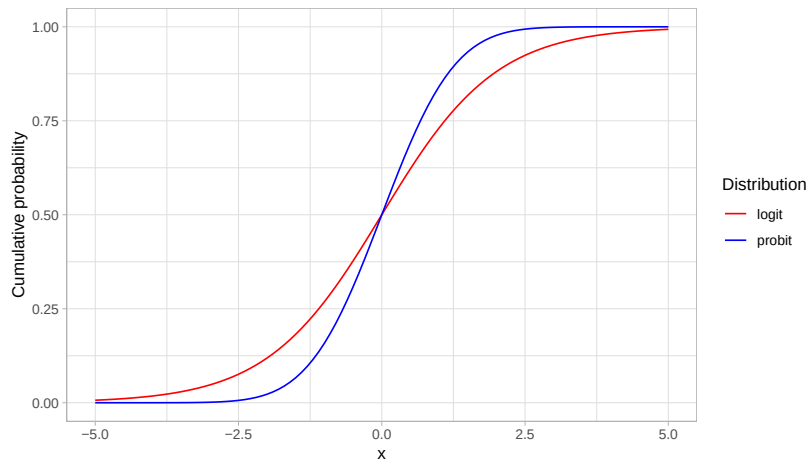
Practical Output: For most datasets, the predicted probabilities and marginal effects of logit and probit are almost identical.

Ease of Use: Logit is more popular due to mathematical simplicity and intuitive odds ratios.

Advanced Modeling: Probit handles certain complex error structures (like varying error variances) more flexibly in econometrics.

You learned about probit and logistic regression. In figure are a logit and probit curve. The x-axis is the input value (i.e., model input x) and the y-axis is the probability corresponding to the input.

Probit versus logit (cont.)



Which of the following is NOT true about about the models.

- a. Both models have a link function.
- b. The probit model has a wider tail than the logit.
- c. The logit model has a wider tail than the probit.
- d. The logit model is the default for a binomial family in R.

Answer: ✓ b.

During this exercise, you will fit a probit and logit model to the Pittsburgh bus data. There are multiple purposes for this and the following exercises:

- ▶ Learning how to change link functions.
- ▶ Learning to notice differences and similarities between using the logit and probit models.
- ▶ Learning how the link functions differ in code.

First, let's code some models! Then, you'll examine the outputs.

```
# Fit a GLM with a logit link and save it as bus_logit
bus_logit <- glm(
  formula = Bus ~ CommuteDays,
  data = bus,
  family = binomial(link = "logit")
)

# Fit a GLM with probit link and save it as bus_probit
bus_probit <- glm(
  formula = Bus ~ CommuteDays,
  data = bus,
  family = binomial(link = "probit")
)

# Print model summaries
summary(bus_logit)
```

```
##
## Call:
## glm(formula = Bus ~ CommuteDays, family = binomial(link = "logit"),
##      data = bus)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.45493    0.11471 -12.683  < 2e-16 ***
## CommuteDays  0.12985    0.02312   5.616 1.96e-08 ***
```

```
summary(bus_probit)
```

```
##
## Call:
## glm(formula = Bus ~ CommuteDays, family = binomial(link = "probit"),
##      data = bus)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.88951    0.06833 -13.017  < 2e-16 ***
## CommuteDays  0.07810    0.01380   5.658 1.53e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 19568  on 15891  degrees of freedom
## Residual deviance: 19536  on 15890  degrees of freedom
## AIC: 19540
##
## Number of Fisher Scoring iterations: 4
```

Question

Do the results differ?

Possible answers

- a. The results are identical.
- b. The coefficient estimates for CommuteDays differ, but a statistically significant relationship exists with both models.
- c. The coefficient estimates for CommuteDays differ, but a statistically significant relationship does not exist with both models.

Answer: ✓ b.

Simulating a logit

Simulations can help us to understand distributions, evaluate our models, and compare study designs. During this exercise, you will simulate a logit distribution. This will allow you to generate data with known parameters. This can be helpful when testing models or comparing study designs (e.g. how many samples do we need to collect?).

```
# Convert from the logit scale to a probability  
# Using the plogis() function, convert 0 on the logit scale to a probability.  
↵  
# Save the output as p.  
p <- plogis(0)  
  
# Simulate a logit  
# Using the rbinom function,  
# generate 10 samples (n = 10) with a size of 1 (size = 1) using the  
↵ probability p.  
  
rbinom(n = 10, size = 1, prob = p)  
  
## [1] 1 0 1 1 1 1 1 0 0 0
```

Simulating a probit

During the previous exercise, you simulated a logit. During this exercise, you will simulate a probit. First, you will convert from the probit scale to a probability. Second, you will use this probability to simulate from a binomial distribution.

```
# Convert from the probit scale to a probability  
# Using function pnorm() to convert 0 on the probit scale to a probability.  
# Save the output as p.  
p <- pnorm(0)
```

```
# Simulate a probit  
# Using the rbinom function,  
# generate 10 samples (n = 10) with a size of 1 (size = 1) using the  
↪ probability p.  
rbinom(n = 10, size = 1, prob = p)
```

```
## [1] 0 1 0 0 1 1 0 0 1 0
```

Section 9

Interpreting and visualizing GLMs

You learned about the Poisson link function. For example, an intercept only equation might be $y \sim \text{Poisson}(\beta)$, How do you convert β , which is on the Poisson link scale, to the raw data scale?

- a. The conversion is not necessary.
- b. Exponentiate β (i.e., `exp(beta)` in R or e^β).
- c. Take the natural log of β (i.e., `log(beta)` in R or $\ln(\beta)$ algebraically)

Answer: ✓ b.

A linear model and Poisson regression have been fit for you. In this exercise, you'll extract the coefficients for both models and compare them. In Step 3, you will need to exponentiate the results from the Poisson regression. This is needed because the Poisson coefficients are estimated on link-scale, not the raw data-scale.

Recall from the video that linear models intercepts are additive whereas Poisson regression intercepts are multiplicative.

```
# Extract the coefficients from lm_out
```

```
lm_coef <- coef(lm_out)  
lm_coef
```

```
## (Intercept)      Month2      Month3      Month4      Month5      Month6  
## 0.12947658 -0.03803116 -0.07840132 -0.05725436 -0.03270239 -0.04336547  
##      Month7      Month8      Month9      Month10     Month11     Month12  
## -0.00582067 -0.05151959 -0.02392103 -0.05420777 -0.02392103 -0.02291921
```

```
# Extract the coefficients from poisson_out
```

```
poisson_coef <- coef(poisson_out)  
poisson_coef
```

```
## (Intercept)      Month2      Month3      Month4      Month5      Month6  
## -2.04425523 -0.34775767 -0.93019964 -0.58375226 -0.29111968 -0.40786159  
##      Month7      Month8      Month9      Month10     Month11     Month12  
## -0.04599723 -0.50734279 -0.20426264 -0.54243411 -0.20426264 -0.19481645
```

```
# Take the exponential using exp()
```

```
poisson_coef_exp <- exp(poisson_coef)  
poisson_coef_exp
```

```
## (Intercept)      Month2      Month3      Month4      Month5      Month6  
## 0.1294766 0.7062700 0.3944749 0.5578014 0.7474262 0.6650709  
##      Month7      Month8      Month9      Month10     Month11     Month12  
## 0.9550446 0.6020933 0.8152482 0.5813315 0.8152482 0.8229857
```

The intercepts are essentially the same, but the other coefficients are different. This is because the Poisson coefficients are multiplicative. Notice that $0.129 * 0.706 = 0.091$ from the Poisson coefficients is the same as $0.129 - 0.038 = 0.091$ from the linear model.

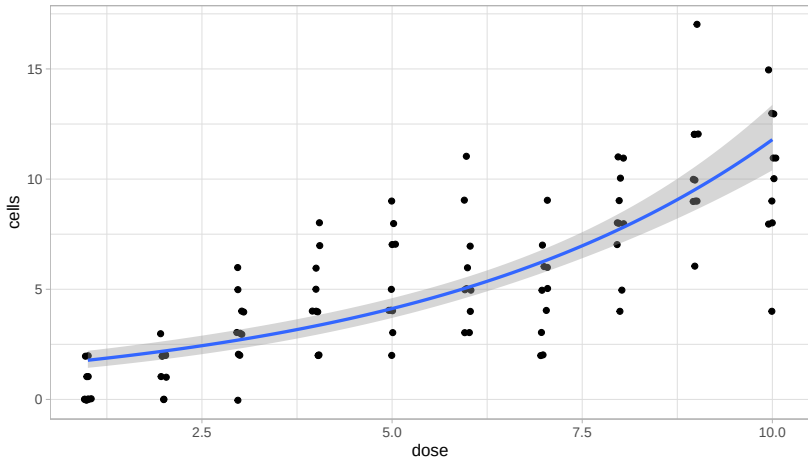
Plotting Poisson regression

Using `geom_smooth` to plot data from a Poisson regression is similar to plotting a binomial. However, you need to change the family argument in `geom_smooth()`. During this exercise, you will plot the number of cancer cells per cm and use a `geom_smooth()`. This is the same simulated dataset you saw in the video.

```
dose = rep(1:10, each = 10)
cells = c(
  1,0,0,0,0,2,0,1,2,0,
  3,0,2,2,1,0,1,2,2,2,
  2,3,5,3,0,3,6,2,4,4,
  2,2,8,4,4,4,7,2,6,5,
  2,5,8,4,7,4,4,7,9,3,
  6,7,9,5,3,5,5,3,4,11,
  2,7,9,3,4,2,6,5,5,6,
  4,5,8,10,11,9,8,8,11,7,
  10,12,9,12,10,12,9,17,6,9,
  15,11,11,10,4,9,13,8,8,13
)
dat <- data.frame(dose, cells)
```

```
# Use geom_smooth to plot a continuous predictor variable
```

```
ggplot(data = dat, aes(x = dose, y = cells)) +  
  geom_jitter(width = 0.05, height = 0.05) +  
  geom_smooth(method = 'glm', method.args = list(family = 'poisson')) +  
  theme_light()
```



Extracting and interpreting odds-ratios

During this exercise, you are going to extract out the odds-ratio ratio from a model that predicts the chance of riding the bus based upon number of commuting days. The resulting odds-ratio (indicated by the CommuteDays term) will give you the change in odds for each extra day a person commutes. For example, an odds-ratio of 1.5 would mean each extra-day a person commutes per week, the odds increase 50% ($1.5 - 1.0 = 0.5 = 50\%$) they will ride the bus.

A logistic regression, bus_out has been fit for you.

```
# Extract out the coefficients
coef_out <- coef(bus_out)

# Convert the coefficients to odds-ratios
exp(coef_out)
```

```
## (Intercept) CommuteDays
## 0.2334164 1.1386623
```

Interpretation

Your odds-ratio was greater than one, so commuting more days increased the chance of riding the bus.

Odds-ratios & confidence intervals in the Tidyverse

The broom package in the Tidyverse give us a powerful tool, `tidy()`, for extracting coefficients and making them readable. More specifically, it allows us to extract the coefficients, as well as the confidence intervals, and exponentiate the outputs. Confidence intervals are another approach for statistical inference. If the confidence intervals for odds-ratios do not include 1, the corresponding coefficient is statistically different than 1. During this exercise, you will use `tidy()` to extract the 95% confidence intervals from the bus model in the previous exercises.

```
# Exponentiate the results and extract the confidence intervals of bus_out  
↪ with tidy()  
tidy(bus_out, exponentiate = TRUE, conf.int = TRUE)
```

```
## # A tibble: 2 x 7  
##   term          estimate std.error statistic  p.value  conf.low  conf.high  
##   <chr>          <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>  
## 1 (Intercept)    0.233    0.115   -12.7  7.32e-37    0.186    0.292  
## 2 CommuteDays    1.14     0.0231    5.62  1.96e- 8    1.09     1.19
```

Question

Does adding an extra commuting day significantly change the odds-ratio for riding the bus?

Answer: The confidence interval for `CommuteDays` does not include 1.0 so the results are statistically significant.

Section 10

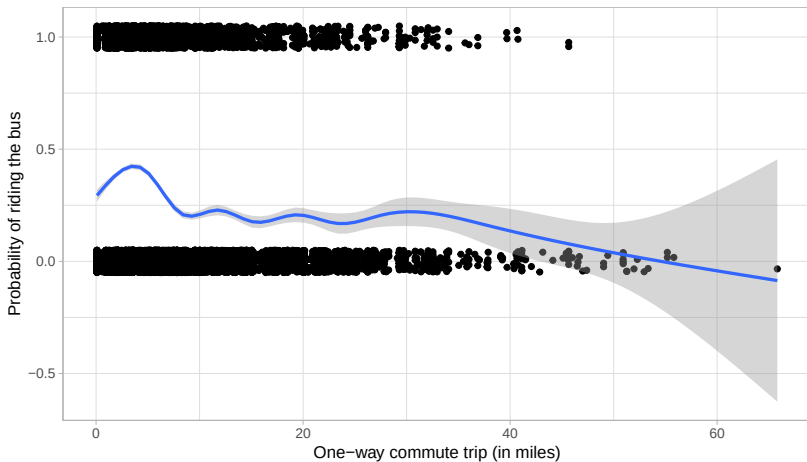
ggplot2 and binomial GLM

Default trend lines

ggplot2 gives us powerful tools to visualize our data and “see” what is going on. However, sometimes we need to wrangle our data to give us what we need. During this exercise, you will look and see if the length of commutes changes the chance somebody rides the bus. You will do this by changing the points to be non-overlapping (or jittered) using `geom_jitter()` and adding smoothed trend line with `geom_smooth()`.

```
bus <- bus %>%  
  mutate(Bus2 = case_when(  
    Bus == "Yes" ~ 1,  
    TRUE ~ 0  
  ))  
  
# Create a jittered plot of MilesOneWay vs Bus2 using the bus dataset  
gg_jitter <- ggplot(data = bus, aes(x = MilesOneWay, y = Bus2)) +  
  geom_jitter(width = 0, height = 0.05) +  
  ylab("Probability of riding the bus") +  
  xlab("One-way commute trip (in miles)")
```

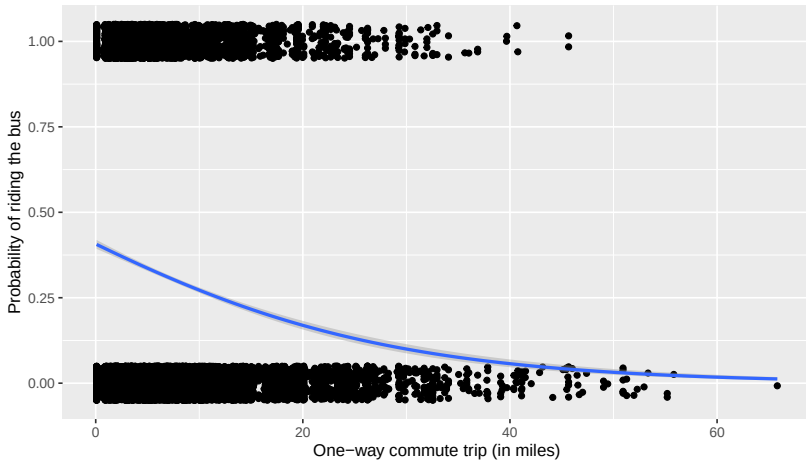
```
# Add a geom_smooth() to your plot  
gg_jitter + geom_smooth() +  
  theme_light()
```



During the previous exercise, you used `ggplot2`'s default `geom_smooth()`. During this exercise, you will use a `glm()` instead. This will allow you to “see” a logistic regression with `ggplot2`. Specifically, you plot the probability someone takes the bus given their commute distance.

You will need to tell `geom_smooth()` to use the `glm()` method. Recall from Chapter 2 that the default family for a `glm()` is a Gaussian family, which produces the same results as a `lm()`. Thus, you will also need to specify the method argument from `glm()`. The code for creating `gg_jitter`, which you built in the last exercise, has been provided for you.

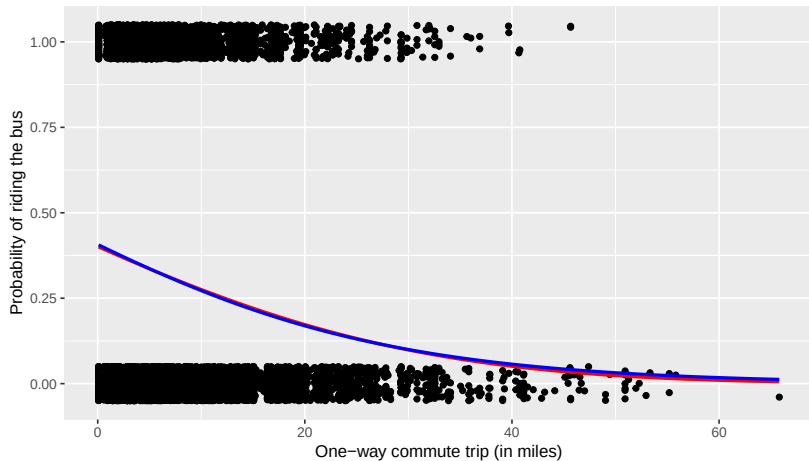
```
# Add a geom_smooth() that uses a GLM method to your plot
gg_jitter + geom_smooth(
  method = "glm",
  method.args = list(family = 'binomial')
)
```



Comparing probits and logits

```
# Add geom_smooth() lines for the probit and logit link functions
gg_jitter +
  geom_smooth(method = 'glm',
              method.args = list(family = binomial(link = 'probit')),
              color = 'red', se = FALSE) +
  geom_smooth(method = 'glm',
              method.args = list(family = binomial(link = 'logit')),
              color = 'blue', se = FALSE)
```

Comparing probits and logits (cont.)



Notice how probits and logit curves are often similar. Hence, the choice of using either of the models usually does not change the results. The main purpose of this exercise is to show you how to change link functions.

Section 11

Multiple Logistic Regression

Fitting a multiple logistic regression

```
# Build a logistic regression with Bus predicted by CommuteDays and  
↪ MilesOneWay  
bus_both <- glm(  
  formula = Bus ~ CommuteDays + MilesOneWay,  
  data = bus,  
  family = 'binomial'  
)
```

```
# Look at the summary of the output  
summary(bus_both)
```

```
##  
## Call:  
## glm(formula = Bus ~ CommuteDays + MilesOneWay, family = "binomial",  
##      data = bus)  
##  
## Coefficients:  
##              Estimate Std. Error z value Pr(>|z|)  
## (Intercept) -0.707515   0.119719  -5.910 3.42e-09 ***  
## CommuteDays  0.066084   0.023181   2.851  0.00436 **  
## MilesOneWay -0.059571   0.003218 -18.512 < 2e-16 ***  
## ---
```

Fitting a multiple logistic regression (cont.)

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 19568  on 15891  degrees of freedom
## Residual deviance: 19137  on 15889  degrees of freedom
## AIC: 19143
##
## Number of Fisher Scoring iterations: 4
```

Building two models

In chapter 2, you built a model to examine if the number of commute days impacted chance of riding the bus. In chapter 3, you visualized if the distance a commuter traveled impacted the chance of riding the bus. During this exercise, you will build both of these logistic regressions using the `glm()` function with the `binomial` family. The bus data frame has been loaded for you.

```
# Build a logistic regression with Bus predicted by CommuteDays
```

```
bus_days <- glm(  
  formula = Bus ~ CommuteDays,  
  data = bus,  
  family = 'binomial'  
)
```

```
# Build a logistic regression with Bus predicted by MilesOneWay
```

```
bus_miles <- glm(  
  formula = Bus ~ MilesOneWay,  
  data = bus,  
  family = 'binomial'  
)
```

```
summary(bus_days)
```

```
##
## Call:
## glm(formula = Bus ~ CommuteDays, family = "binomial", data = bus)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.45493    0.11471 -12.683  < 2e-16 ***
## CommuteDays  0.12985    0.02312   5.616 1.96e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 19568  on 15891  degrees of freedom
## Residual deviance: 19536  on 15890  degrees of freedom
## AIC: 19540
##
## Number of Fisher Scoring iterations: 4
```

```
summary(bus_miles)
```

```
##
## Call:
## glm(formula = Bus ~ MilesOneWay, family = "binomial", data = bus)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.375824   0.027801  -13.52   <2e-16 ***
## MilesOneWay -0.060760   0.003197  -19.01   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 19568  on 15891  degrees of freedom
## Residual deviance: 19145  on 15890  degrees of freedom
## AIC: 19149
##
## Number of Fisher Scoring iterations: 4
```

The single and multiple regression models estimated different coefficient values. This is why multiple regression model results often are described with 'corrected for'. For example, here you might describe the probability of riding the bus based upon commute days, corrected for the commute distance.

Comparing variable order

The order of predictor variables can be important, especially if predictors are correlated. This is because changing the order of correlated predictor variables can change the estimates for the regression coefficients. The name for this problem is Multicollinearity.

During this exercise, you will build two different multiple regressions with the bus data in order to compare the importance of model inputs order. First, examine the correlation between `CommuteDays` and `MilesOneWay`. Second, build two logistic regressions using the bus data frame where `Bus` is predicted by `CommuteDays` and `MilesOneWay` in separate orders. After you build the two models, look at each model's `summary()` to see the outputs.

```
# Run a correlation  
cor(bus$CommuteDays, bus$MilesOneWay)
```

```
## [1] -0.1378974
```

Build a glm with CommuteDays first and MilesOneWay second

```
bus_one <- glm(  
  formula = Bus ~ CommuteDays + MilesOneWay,  
  data = bus,  
  family = 'binomial'  
)
```

Build a glm with MilesOneWay first and CommuteDays second

```
bus_two <- glm(  
  formula = Bus ~ MilesOneWay + CommuteDays ,  
  data = bus,  
  family = 'binomial'  
)
```

```
# Print model summaries
```

```
summary(bus_one)
```

```
##  
## Call:  
## glm(formula = Bus ~ CommuteDays + MilesOneWay, family = "binomial",  
##      data = bus)  
##  
## Coefficients:  
##              Estimate Std. Error z value Pr(>|z|)  
## (Intercept) -0.707515   0.119719  -5.910 3.42e-09 ***  
## CommuteDays  0.066084   0.023181   2.851 0.00436 **  
## MilesOneWay -0.059571   0.003218 -18.512 < 2e-16 ***  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for binomial family taken to be 1)  
##  
##      Null deviance: 19568  on 15891  degrees of freedom  
## Residual deviance: 19137  on 15889  degrees of freedom  
## AIC: 19143  
##  
## Number of Fisher Scoring iterations: 4
```

```
summary(bus_two)
```

```
##
## Call:
## glm(formula = Bus ~ MilesOneWay + CommuteDays, family = "binomial",
##      data = bus)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.707515   0.119719  -5.910 3.42e-09 ***
## MilesOneWay -0.059571   0.003218 -18.512 < 2e-16 ***
## CommuteDays  0.066084   0.023181   2.851  0.00436 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 19568  on 15891  degrees of freedom
## Residual deviance: 19137  on 15889  degrees of freedom
## AIC: 19143
##
## Number of Fisher Scoring iterations: 4
```

Notice how in this case, order was not important for the regression predictor variables. Usually, this is the case. Practically speaking, the easiest way to check this assumption to change the order of the variables in the regression.

Section 12

Formulas in R

You learned about `model.matrix()` and its relationship to `glm()`. Now, you can “peek under the hood” of R and see how formulas work. The input to `model.matrix()` is similar to formula inputs of `lm()` and `glm()` with one key difference: `model.matrix()` does not have a left-hand side (e.g. `y ~`), only the right-hand side (e.g., `~ x`).

For example, `model.matrix( ~ x1)` or `model.matrix( ~ x1 + x2)` both would be valid inputs. The output from `model.matrix()` is called a prediction matrix because it is the prediction (or right-hand side) of a formula in R.

```
size <- c(1.1, 2.2, 3.3)
count <- c(10, 4, 2)

# Use model.matrix() with size
model.matrix(~ size)

##      (Intercept) size
## 1             1  1.1
## 2             1  2.2
## 3             1  3.3
## attr(,"assign")
## [1] 0 1
```

```
# Use model.matrix() with size and count
model.matrix(~ size + count)
```

```
##      (Intercept) size count
## 1             1  1.1    10
## 2             1  2.2     4
## 3             1  3.3     2
## attr("assign")
## [1] 0 1 2
```

Notice how `model.matrix()` includes an intercept for the design matrix. This means all observations would share the same intercept estimate. Next, you'll see how to specify intercepts.

In the previous exercise, you saw how to code multiple slopes. During this exercise, you will get to explore intercepts with `model.matrix()` using a simple vectors of colors. First, build a matrix including a reference intercept. This input only has the intercept variable's name (e.g. `~x1`). Second, build a matrix including an intercept for each color. This input has the intercept variable's name and `-1` (e.g. `~x1 - 1`).

```
color <- c("red", "blue", "green")  
  
# Create a matrix that includes a reference intercept  
model.matrix(~ color)
```

Intercepts (cont.)

```
## (Intercept) colorgreen colorred
## 1          1          0          1
## 2          1          0          0
## 3          1          1          0
## attr(,"assign")
## [1] 0 1 1
## attr(,"contrasts")
## attr(,"contrasts")$color
## [1] "contr.treatment"
```

```
# Create a matrix that includes an intercept for each group  
model.matrix(~ color-1)
```

```
##   colorblue colorgreen colorred  
## 1         0         0         1  
## 2         1         0         0  
## 3         0         1         0  
## attr("assign")  
## [1] 1 1 1  
## attr("contrasts")  
## attr("contrasts")$color  
## [1] "contr.treatment"
```

Notice how `model.matrix()` has different outputs for each input. The first input gives a baseline intercept and contrast. The second input gives an intercept for each group. The first allows you to compare groups to a reference group and the second allows you to compare all groups to zero.

With multiple intercepts, predictor variable order become important if you want to estimate intercepts for one group. This is because R only estimates intercepts for the first predictor variable. All other predictor intercepts will contrast to a reference group within the factor.

In the previous exercise, you constructed a contrast matrix with one intercept with and without an intercept for each group. During this exercise, build a contrast matrix with two group variables and use the -1 option. Then, swap the order of the input variables. This allows you to see how order is important for the contrast levels.

```

shape <-c( "square", "square", "circle")

# Create a matrix that includes color and then shape
model.matrix(~ color + shape - 1)

##      colorblue colorgreen colorred shapesquare
## 1           0           0           1           1
## 2           1           0           0           1
## 3           0           1           0           0
## attr("assign")
## [1] 1 1 1 2
## attr("contrasts")
## attr("contrasts")$color
## [1] "contr.treatment"
##
## attr("contrasts")$shape
## [1] "contr.treatment"

```

```
# Create a matrix that includes shape and then color  
model.matrix(~ shape + color -1)
```

```
##   shapecircle shapesquare colorgreen colorred  
## 1           0           1           0           1  
## 2           0           1           0           0  
## 3           1           0           1           0  
## attr("assign")  
## [1] 1 1 2 2  
## attr("contrasts")  
## attr("contrasts")$shape  
## [1] "contr.treatment"  
##  
## attr("contrasts")$color  
## [1] "contr.treatment"
```

Assumptions of Multiple Logistic Regression

Simpson's paradox occurs when adding or removing a coefficient changes the results of analysis and is important for regressions.

The 1973 Graduate School admission data from UC-Berkeley illustrates this point. At first glance, it appears females are less likely to be admitted to graduate programs. However, including Department as a coefficient causes gender's significance to disappear. As it turns out, prospective female students applied to more competitive programs than males.

Data note: When looking at the data, you are provided with four columns: Dept, Gender, Admitted, and Rejected. You can build a "binomial" `glm()` by binding the Admitted and Rejected columns.

```
Dept = c("A", "A", "B", "B", "C", "C", "D", "D", "E", "E", "F", "F")
Gender = c("Male", "Female", "Male", "Female", "Male", "Female",
           "Male", "Female", "Male", "Female", "Male", "Female")
Admitted = c(512, 89, 353, 17, 120, 202, 138, 131, 53, 94, 22, 24)
Rejected = c(313, 19, 207, 8, 205, 391, 279, 244, 138, 299, 351, 317)

UCB_data <- data.frame(Dept, Gender, Admitted, Rejected)
```

```
# Build a binomial glm where Admitted and Rejected are predicted by Gender
glm_1 <- glm(
  formula = cbind(Admitted, Rejected) ~ Gender,
  data = UCB_data,
  family = "binomial"
)
```

```
# Build a binomial glm where Admitted and Rejected are predicted by Gender
↪ and Dept
glm_2 <- glm(
  formula = cbind(Admitted, Rejected) ~ Gender + Dept,
  data = UCB_data,
  family = "binomial"
)
```

```
# Look at the summary of both models
summary(glm_1)
```

```
##
## Call:
## glm(formula = cbind(Admitted, Rejected) ~ Gender, family = "binomial",
##      data = UCB_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.83049    0.05077  -16.357   <2e-16 ***
## GenderMale   0.61035    0.06389   9.553   <2e-16 ***
```

```
summary(glm_2)
```

```
##
## Call:
## glm(formula = cbind(Admitted, Rejected) ~ Gender + Dept, family =
  ↪ "binomial",
##     data = UCB_data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.68192    0.09911   6.880 5.97e-12 ***
## GenderMale  -0.09987    0.08085  -1.235   0.217
## DeptB       -0.04340    0.10984  -0.395   0.693
## DeptC      -1.26260    0.10663 -11.841 < 2e-16 ***
## DeptD      -1.29461    0.10582 -12.234 < 2e-16 ***
## DeptE      -1.73931    0.12611 -13.792 < 2e-16 ***
## DeptF      -3.30648    0.16998 -19.452 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 877.056  on 11  degrees of freedom
## Residual deviance:  20.204  on  5  degrees of freedom
## AIC: 103.14
##
## Number of Fisher Scoring iterations: 4
```

Question

Which of the following statements is true about the regression coefficients?

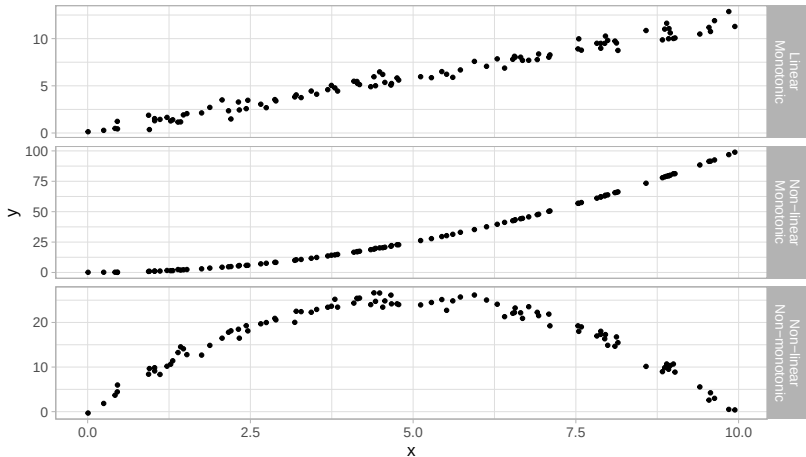
- a. Adding Dept had no effect on the outputs.
- b. All of the Dept coefficients are significantly different from each other.
- c. Adding Dept causes gender to not be a significant coefficient.
- d. Gender is not significant in either model.

Answer: ✓ c.

Notice how adding the department made Gender no longer a significant predictor variable. In this case, the academic departments are important. If you were to dig into the data further, you would find different departments had different rejection rates and also that applications by gender were not balanced across departments.

You explored the distance commuters traveled and the linear effect this had on the probability of somebody riding the bus. However, what if this relationship is non-linear and non-monotonic?

```
ggplot(dat, aes(x, y)) +
  geom_point(size = 1) +
  facet_wrap(~type, ncol = 1, strip.position = "right", scales = "free_y") +
  labs(x = "x", y = "y") +
  theme_light()
```



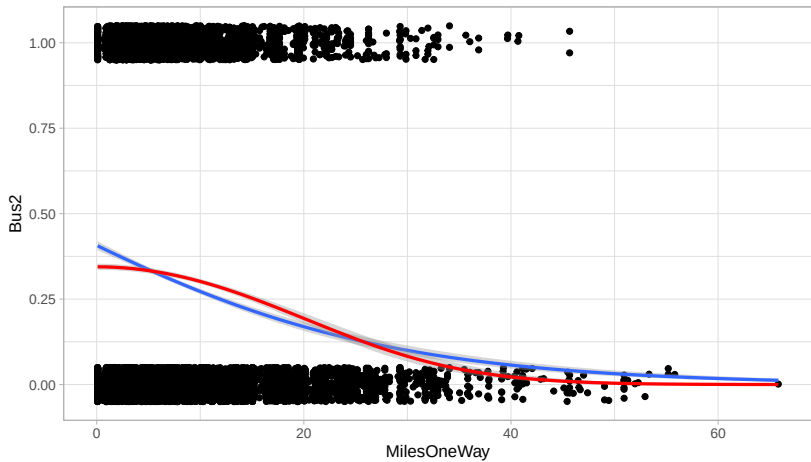
For example, what if people who commute the shortest distances and longest are less likely to ride the bus? You can add non-linear terms to formulas in R using the `I(...)` function as part of your formula. For example `y~I(x^2)` allows you to estimate a coefficient for $x*x$. During this exercise, you will examine the bus data more.

```
# Plot linear effect of travel distance on probability of taking the bus  
gg_jitter <-
```

```
  ggplot(data = bus, aes(x = MilesOneWay, y = Bus2)) +  
  geom_jitter(width = 0, height = 0.05) +  
  geom_smooth(method = 'glm',  
              method.args = list(family = 'binomial'))
```

```
# Add a non-linear equation to a geom_smooth()
```

```
p <- gg_jitter +  
  geom_smooth(method = 'glm',  
              method.args = list(family = 'binomial'),  
              formula = y ~ I(x^2),  
              color = 'red')
```



Based upon the extra plot, do you think the data is linear or non-linear? The two lines are similar to each other. However, with the non-linear (red) line, the probability of commuting stays fairly constant for commutes < 10 miles. Above this distance, the probability drops off. This suggests a non-linear trend in the data and the need for more investigation or insight for your client.

Section 13

Key Takeaways

Key Takeaways

1. **GLMs extend linear models** by allowing non-normal error distributions.
2. **Poisson regression** models counts — coefficients are on the **log scale** (exponentiate them!).
3. **Logistic regression** models binary outcomes — coefficients are **log-odds** (exponentiate $\rightarrow$ odds ratios).
4. **Probit vs Logit** give nearly identical results; logit is more common due to interpretability.
5. **Multiple regression** can reveal **Simpson's paradox** — always check for confounding variables.
6. Use `glm()`, `summary()`, `tidy()`, `predict()`, and `geom_smooth()` for a complete workflow.

Useful R Functions Recap

Task	Function
Fit GLM	<code>glm(formula, family = ...)</code>
Summary	<code>summary()</code> , <code>tidy()</code>
Coefficients	<code>coef()</code> , <code>exp(coef())</code>
Confidence intervals	<code>confint()</code>
Predictions	<code>predict(..., type = "response")</code>
Plot	<code>geom_smooth(method = "glm")</code>

Thank You